Computer Networks 258 (2025) 111026

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Energy-latency tradeoff for task offloading and resource allocation in vehicular edge computing

Yuxuan Long, Zhenyu Wang *, Shizhan Lan, Rui Zhang, Kai Xu

School of software, South China University of Technology, Guangzhou, China

ARTICLE INFO

Keywords:
Edge computing
Vehicular network
Task offloading
Convex optimization

ABSTRACT

Vehicular edge computing (VEC) has emerged as a cutting-edge distributed computing paradigm capable of addressing network congestion and excessive energy use in vehicular systems. To enhance VEC performance, we examined the energy–latency tradeoff for partial tasks offloading in end-VEC-cloud orchestrated networks. We formulated a joint computation offloading and resource allocation problem aimed at minimizing latency and energy consumption. To address the underlined problem, we proposed a collaborative task splitting and resource allocation optimization (CTSRAO) algorithm. We initially decoupled the problem into two convex sub-problems and then applied the Lagrangian and simplex methods for joint optimization of computation resources and task splitting ratio. Furthermore, we investigated the criteria for determining whether a task should be offloaded to the VEC or cloud. Simulation results showed that our algorithm significantly enhances systems performance, achieving lower latency and energy consumption than the benchmark and state-of-the-art methods.

1. Introduction

With the advancement of vehicular networks and 5G wireless systems, numerous latency-sensitive and computation-intensive on-vehicle applications, including virtual/augmented reality, automatic driving, and infotainment services [1], have presently started demanding more computational resources and bandwidth. However, the limited computational and communication capacities of smart vehicles cannot meet these demands [2]. To address this issue, mobile edge computing (MEC) has recently emerged as an effective solution, deploying cloud computing resources at the network edge to efficiently support latency-sensitive applications.

Vehicular edge computing (VEC) is an emerging distributed computing paradigm that integrates mobile edge computing with vehicular networks, extending computation and communication capabilities to the vehicular network edge [3]. This technology enhances the availability of computation and caching resources at the edge of vehicle networks. It enables on-vehicle tasks to be processed locally on the VEC servers rather than relying on cloud data centers, significantly reducing the mobile data traffic in corn networks. In contrast to traditional MEC, VEC faces challenges due to the fast-moving vehicles, leading to frequent topology changes and an unstable communication environment [4]. Additionally, the limited computational resources of VEC make it challenging to meet the latency requirements for all computation-intensive tasks [5]. The insufficient energy supply at the network edge also poses a risk, as lightweight VEC nodes may deplete energy, potentially causing data loss or other serious consequences.

To address the above challenges, the study focuses on optimizing resource allocation and task offloading by integrating cloud and edge computing to reduce systems latency [6]. Concurrently, other researchers aim to balance workloads to minimize overall energy consumption [7]. However, these scholars typically address the latency or energy consumption in isolation, neglecting their interaction [8]. For instance, increasing computing resources for a task reduces the latency but increases the energy consumption. Consequently, exploring the tradeoff between latency and energy consumption is crucial.

We address the energy–latency tradeoff in an end-VEC-cloud orchestrated network, formulating it as a system utility maximization problem. We decouple this issue into two sub-problems. The first one optimizes the task-splitting ratio between VEC and cloud, which is a linear programming problem; the second one optimizes resource allocation between VEC and cloud, which is a convex problem. The Lagrangian multiplier method and gradient method solve these sub-problems. Based on the solutions, we propose a collaborative optimization algorithm for task-splitting and resource allocation that significantly reduces system latency and energy consumption. Additionally, we explore scenarios involving complete task offloading. The main contributions of our research are summarized as follows:

* Corresponding author.
E-mail address: wangzy@scut.edu.cn (Z. Wang).

https://doi.org/10.1016/j.comnet.2024.111026
Received 7 April 2024; Received in revised form 31 October 2024; Accepted 26 December 2024
Available online 2 January 2025

Table 1
Main notations.

Notation	Explanation
V_i^{vec}	Vehicle i transmission rate.
P_i	Vehicle i transmission.
T_i^{vec}	Vehicle i latency for transmitting partial task to VEC.
S_i	Vehicle i task size.
α_i	Vehicle i partial task processed on VEC.
β_i	Vehicle i partial task processed on cloud.
T_i^c	Latency of VEC for transmitting vehicle i to cloud.
e_i^v	Vehicle i transmission energy consumption.
e^c	VEC transmission energy consumption.
ξ^v, ξ^c	Vehicle and VEC energy factors.
D_i^{vec}, D_i^c, D_i^v	Computation latency of VEC, cloud, and vehicle.
f_i^{vec}, f_i^c, f_i^v	Computation resources allocated to task of vehicle i for VEC, cloud, and vehicle.
F^{vec}, F^c	VEC and cloud computation resources.
Q_i	The computation resources requirement for the task of vehicle i.
P_i^v, P_i^{vec}, P_i^c	The computation energy consumption of vehicles, VEC, and cloud.
ζ^v, ζ^{vec}, ζ^c	The computation energy factors of vehicle, VEC, and cloud.
T_i^{tol}	Vehicle i total latency.
E_i^{tol}	Vehicle i total energy consumption.
T_i^{max}	Maximum tolerable latency.
E_i^{max}	Maximum tolerable energy consumption.
U_i	The system utility.

(1) We introduce a novel end-VEC-cloud orchestrated framework that enables parallel computation of vehicle tasks across local, VEC, and cloud resources.
(2) We analyze the tradeoff between latency and energy consumption in end-VEC-cloud orchestrated networks, optimizing the computational resource allocation between VEC, cloud, and vehicle transmission power. Additionally, we optimize the computation offloading ratio of the VEC and cloud, establishing the necessary conditions required for complete task offloading.
(3) We perform simulations to demonstrate the effectiveness of the proposed algorithm under various parameters, evaluating the results through comparative experiments.

The rest of the paper is structured as follows. Section 2 reviews related work on latency-energy tradeoffs. Section 3 introduces the VEC systems model. Section 4 details the latency-energy tradeoff problem formulation, and presents the CTSRAO algorithm. Section 5 presents the simulation results. Section 6 concludes the findings of this study (see Table 3).

2. Related work

To cope with a larger amount of mobile data traffic in vehicular networks, VEC emerges as a promising technique to enhance vehicular networks performance by extending computation resources to the edge of networks [9]. This emerging technology alleviates network traffic pressure by integrating MEC into the vehicular networks [10]. Literature [11] illustrates VEC comprehensively, including VEC architecture, VEC technical issues and current challenges. Literature [12] presents a comprehensive review of state-of-the-art researches on VEC. Recent researches on VEC related to task offloading can be classified into three categories based on optimization object: latency minimization, energy consumption, latency-energy tradeoff.

2.1. Latency minimization

Latency directly impacts the service quality, making its minimization a fundamental metric in VEC research. Zhao et al. [13] examined cloud-assisted VEC for optimizing computation offloading. They proposed a collaborative scheme that splits the optimization problem into two sub-problems underlining computation offloading decision-making and resource allocation. Ren et al. [6] analyzed the partial offloading and resource allocation between edge and cloud, converting these two into an equivalent convex problem to minimize latency, which is solved using Karush–Kuhn–Tucker (KKT) conditions. Sun et al. [14] explored a VEC-cloud-assisted offloading scenario and modeled it as an NP-hard scheduling problem, proposing a genetic-based heuristic algorithm for its resolution. Li et al. [15] incorporated software-defined networks (SDN) into vehicular networks and presented an SDN-based task offloading problem in VEC to minimize total processing latency.

2.2. Energy consumption minimization

Energy consumption becomes a critical issue in VEC networks due to rising electricity costs [16]. Zhou et al. [8] developed a stochastic traffic model for VEC nodes and user equipment (UE), to minimize the overall energy consumption through efficient workload offloading, using an ADMM-based distributed algorithm. Tang et al. [17] introduced a Low-Power Edge Computing System (LoPECS) for real-time autonomous vehicles and robots, engineering a dynamic task offloading algorithm to enhance user experience in autonomous driving. Bahreini et al. [18] propounded an energy-aware resource management problem in VEC as a mixed integer non-linear program, solved by a polynomial-time greedy algorithm. [19] introduces a shared offload strategy that decomposes tasks into independent units to enhance efficiency. It emphasizes using a Deep Reinforcement Learning (DRL) method to minimize energy consumption.

2.3. Energy–latency tradeoff

The energy–latency tradeoff problem is more complex due to computation resources and energy constraints. Zhang et al. [20] propounded an integrated framework for computing offloading and resource allocation in MEC networks, proposing an energy-ware scheme to minimize latency and energy use. Yadav et al. [21] analyzed vehicle mobility functions to optimize offloading failures. They formulated a dynamic computation offloading framework addressing latency and energy constraints. The proposed three-phase ECOS scheme effectively mitigates overload risks and minimizes latency and energy costs. In [22], a novel task offloading framework called joint offloading of tasks and energy (JOTE) was propounded to tackle energy supply and task latency challenges at edge nodes. The Lyapunov-based offloading strategy aims to minimize average task latency and energy consumption. Literature [23] analyzes the delay and cost of computation offloading in VEC and formulates a multi-objective optimization problem to minimize both factors. It propose a particle swarm optimization based computation offloading (PSOCO) algorithm to obtain the Pareto-optimal solutions to the multi-objective optimization problem. Similarly, Qin et al. [24] proposed an optimization framework for

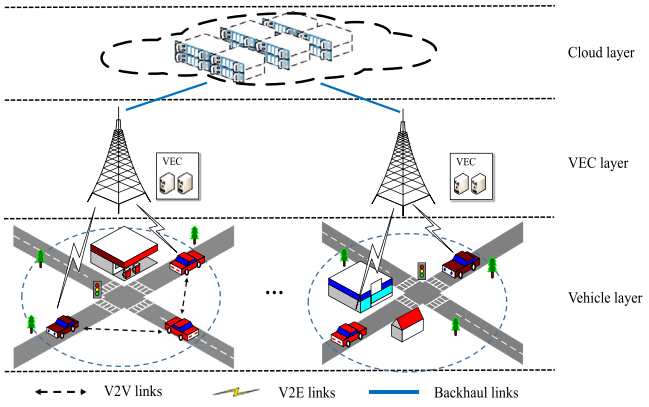


Fig. 1. Structure of the VEC system.

joint resource allocation and tasks partial offloading in MEC-enhanced IoT networks. However, our work extends this framework by tackling the more challenging problem of joint resource allocation and task offloading in end-VEC-cloud orchestrated networks.

3. Systems model and problem formulation

3.1. System model

Fig. 1 illustrates the end-VEC-cloud orchestrated network architecture, which is organized into three vertical layers: vehicle, VEC, and cloud. The vehicle layer consists of local vehicular networks distributed across road sections, generating numerous tasks that can be offloaded to the VEC and cloud layers to reduce latency. Table 1 provides key notations.

(1) Cloud layer:
This topmost layer is an independent cloud computing platform with extensive computing and storage resources. It oversees and manages the operation of VEC nodes from a global perspective.
(2) VEC layer:
This layer features roadside VEC servers equipped with both computation and communication capabilities [25]. Due to their proximity to users, it handles latency-sensitive tasks efficiently. It acts as a conduit between the vehicle and cloud layers, facilitating uplinks to the cloud platform and downlinks to the vehicles. When VEC resources are insufficient, computation-intensive tasks can be offloaded to the cloud layer.
(3) Vehicle layer:
The lowest tier in this architecture comprises smart vehicles equipped with computation and wireless communication devices (3GPP, IEEE 802.11P, Bluetooth) [26]. These vehicles can process local tasks and communicate with other vehicles or VEC servers. Tasks generated by smart vehicles and onboard users can be offloaded to VEC or the cloud to minimize latency.

As modeling like previous studies [27–29], Gaussian distribution is widely adopted in vehicle traffic systems to model the velocity of vehicles. For simplicity, we assume that vehicles is moving at a constant velocity within the coverage of VEC server. Also, to limit the velocity of vehicle in a reasonable range, we assume the velocity is randomly following a truncated Gaussian distribution. The truncated Gaussian probability density function (PDF) is denoted as:

$$f(v_n) = \begin{cases} \dfrac{2e^{\frac{-(v_n-\mu)^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}(erf(\frac{v^{max}-\mu}{\sigma\sqrt{2}})) - erf(\frac{v^{min}-\mu}{\sigma\sqrt{2}})} \\ v^{min} \le v_n \le^{max} \\ 0, \quad otherwise \end{cases}$$

$$exf(v_n) = \frac{2}{\sqrt{\pi}} \int_0^{v_n} e^{-\eta^2} d\eta \tag{1}$$

Where μ is the mean velocity, σ is the standard deviation of the vehicle velocity and $exf(v_n)$ is the Gauss error function.

Each vehicle and VEC server is equipped with a global position system (GPS). The position of vehicle i is denoted as (d_i^x, d_i^y), and the position of the VEC is labeled as (D^x, D^y). Let $\mathbb{L}$ represent the coverage radius of the VEC server. The driving time of vehicle i within the coverage of the VEC server is expressed as follows:

$$t_i = \frac{\sqrt{\mathbb{L}^2 - (d_i^y - D^y)^2} - (d_i^x - D^x)}{v_i} \tag{2}$$

where v_i denotes the velocity of vehicle i. Let $T_i^{max} = t_i$, indicating that offloading must be completed before the vehicle exits the VEC coverage area.

3.2. Transmission model

Assuming transmission interference is negligible due to the use of the orthogonal frequency division multiple access (OFDMA) method, the transmission rate of vehicle i is expressed as:

$$V_i^{vec} = B\log(1 + \frac{P_i C_i}{N}) \tag{3}$$

where B denotes the bandwidth of the VEC server; P_i indicates the transmission power of vehicle i; C_i signifies the channel gain between vehicle i and the VEC server; N signifies the noise. Each vehicle is assumed to have a computation task. The latency for vehicle i to transmit a partial task to the VEC server is expressed as:

$$T_i^{vec} = \frac{(\alpha_i + \beta_i)S_i}{V_i^{vec}} \tag{4}$$

where $\alpha_i, \beta_i \in [0, 1]$ represent the task splitting ratios; S_i signifies the task size of vehicle i; α_i denotes the fraction of vehicle i task that is offloaded to VEC; β_i indicates the fraction offloaded to the cloud. Since VEC servers relay tasks to the cloud, any task portion intended for the cloud must first be transmitted to VEC. To ensure task normalization, we have $\alpha_i + \beta_i \in [0, 1]$ where V_c denotes the transmission rate of VEC. The transmission latency of VEC servers for offloading the task from vehicle i to the cloud is formulated as:

$$T_i^c = \frac{\beta_i S_i}{V_c} \tag{5}$$

3.3. Transmission energy model

As vehicle data uploads to the VEC are instantaneous rather than continuous, uplink transmission energy consumption is dependent on the instantaneous power:

$$e_i^v = \xi^v P_i \tag{6}$$

$$e^c = \xi^c P^c \tag{7}$$

where ξ^v and ξ^c denote energy factors for the vehicle and VEC, respectively, and P^c signifies the transmission power of VEC.

3.4. Computation model

The computational resources of the VEC are described by the following model, where f_i^{vec} (in CPU cycle/s) denotes the computational resources allocated to the task of vehicle i, and Q_i represents the number of CPU cycles needed to compute the underlined task (of vehicle i). For simplicity, we assume that Q_i is proportional to the S_i. Therefore, the computation latency for the task of vehicle i at the VEC and cloud can be expressed as follows:

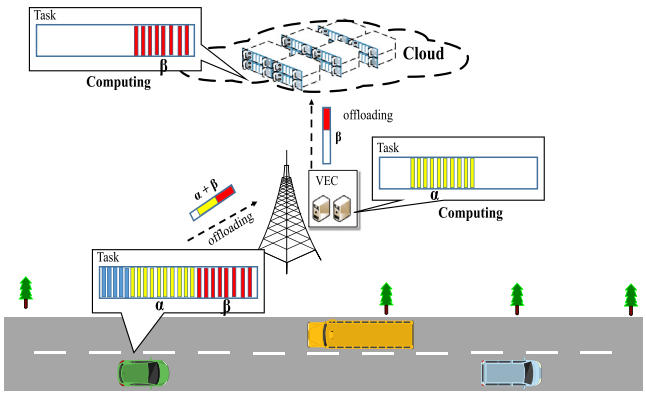


Fig. 2. Task offloading.

$$D_i^{vec} = \frac{\alpha_i Q_i}{f_i^{vec}} \tag{8}$$

$$D_i^{c} = \frac{\beta_i Q_i}{f_i^{c}} \tag{9}$$

The number of CPU cycles for the local computing task of vehicle i is determined by $(1-\alpha_i-\beta_i)Q_i$, assuming the computation resources of smart vehicles are constant at f^v. Consequently, the computation time of the smart vehicle is expressed as:

$$D_i^{v} = \frac{(1-\alpha_i-\beta_i)Q_i}{f^v} \tag{10}$$

3.5. Computation energy model

Typically, the energy consumption power is related to the number of CPU cycles per clock, which is directly proportional to the square of the supply voltage. As noted in [30], the supply voltage is approximately proportional to the number of CPU cycles per clock. Consequently, the computation energy consumption of vehicles, VEC, and cloud can be expressed as follows:

$$\mathcal{P}_i^{v} = (1-\alpha_i-\beta_i)\zeta^v f^v Q_i^2 \tag{11}$$

$$\mathcal{P}_i^{vec} = \alpha_i \zeta^{vec} f^{vec} Q_i^2 \tag{12}$$

$$\mathcal{P}_i^{c} = \beta_i \zeta^{c} f^{c} Q_i^2 \tag{13}$$

where ζ^v, ζ^{vec}, and ζ^c represent the computation energy factor of vehicle, VEC, and cloud, respectively.

3.6. Latency model

As noted above, the computation and communication units of VEC are usually separated, enabling concurrent task processing by both VEC and cloud or locally by smart vehicles. Consequently, the tasks are handled in parallel across smart vehicles, VEC, and the cloud. Fig. 2 illustrates the task offloading process and the interactions between vehicles and VEC.

The smart vehicle transmits a portion of the task $(\alpha+\beta)$ to the VEC. The VEC then computes the task of part α and forwards the task of part β to the cloud simultaneously. The cloud computes the task of part β. Since computation occurs in parallel, the total latency is determined by the maximum latency among the vehicle, VEC, and cloud. The process can be expressed as:

$$T_i^{tol} = \max(D_i^{v}, D_i^{vec} + T_i^{vec}, T_i^{vec} + T_i^{c} + D_i^{c}) \tag{14}$$

3.7. System utility function

To evaluate the quality of service (QoS) in end-VEC-cloud orchestrated networks, this paper introduces a system utility function that considers both total latency and energy consumption. Differing from previous work [31], which focused solely on minimizing system latency, our approach effectively balances energy consumption and latency. We introduce a weight factor $\theta(0<\theta<1)$ in the utility function to adjust the emphasis on latency versus energy use. The function incorporates the maximum tolerable system latency T_i^{max} and energy consumption E_i^{max} for each task i.

$$U_i = \theta(T_i^{max} - T_i^{tol}) + (1-\theta)(E_i^{max} - E_i^{tol}) \tag{15}$$

Both metrics remain constants and define the upper bound of the function, where E_i^{tol} represents total energy consumption for task i, encompassing both transmission and computing energy consumption, detailed as follows:

$$E_i^{tol} = \mathcal{P}_i^{v} + \mathcal{P}_i^{vec} + \mathcal{P}_i^{c} + e_i^{vec} + e^{c} \tag{16}$$

Substituting Eqs. (6), (7), (11), (12), and (13) into Eq. (16):

$$E_i^{tol} = (1-\alpha_i-\beta_i)\zeta^v f^v Q_i^2 + \alpha_i \zeta^{vec} f_i^{vec} Q_i^2 + \beta_i \zeta^c f_i^c Q_i^2 + \xi^v P_i + \xi^c P^c \tag{17}$$

The system utility function effectively represents the performance of end-VEC-cloud orchestrated networks. For instance, the system utility becomes negative if total latency and energy consumption surpass T^{max} and E^{mex}, respectively.

3.8. Problem formulation

The problem can be framed as a joint optimization of resource allocation and task offloading, aiming to maximize system utility. Given the finite computation resources of the VEC and cloud, the problem includes two resource constraints. Additionally, latency and energy consumption constraints are incorporated to ensure system QoS. Therefore, the optimization problem can be expressed as follows:

$$P1: \max_{\alpha_i,\beta_i,f_i^{vec},f_i^{c},P_i} \sum_{i=1}^{n} U_i$$

$$s.t. \quad \sum_{i=1}^{n} f_i^{vec} \le F^{vec} \tag{C1}$$

$$\sum_{i=1}^{n} f_i^{c} \le F^{c} \tag{C2}$$

$$T_i^{tol} \le T_i^{max} \tag{C3}$$

$$E_i^{tol} \le E_i^{max} \tag{C4}$$

$$\alpha+\beta \le 1, \alpha \ge 0, \beta \ge 0 \tag{C5}$$

where (C1) denotes the constraint on total computation resources in the VEC server; (C2) represents the constraint on the cloud's available computation resources; (C3) signifies that total latency must not exceed the maximum tolerable constraint; (C4) indicates the energy constraint for the task of vehicle i; (C5) denotes the task splitting ratio.

The system utility function is non-differentiable with respect to f_i^{vec} $f_i^{c}, P_i, \alpha_i, \beta_i$, resulting in a non-convex and non-smooth optimization problem. Additionally, the optimization variables are highly coupled and involve mixed combinatorial elements, making it difficult to find a globally optimal solution in polynomial time using optimization techniques. To this end, we decouple the original problem into two sub-problems to simplify the complexity. We then convert these sub-problems into convex forms. Finally, we propose CTSRAO, a low-complexity approach based on convex theory, to obtain an approximate optimal solution.

4. Solution

In this section, we decouple the original problem into two sub-problems: task splitting and computation resource allocation. Both sub-problems can be solved using the convex technique.

4.1. Computation resource allocation problem

The original problem can be reformulated as a resource allocation problem by treating variables α_i, and β_i as non-negative constants ($0 \leq \alpha_i, \beta_i \leq 1$), resulting in the following:

$$P1.1: \max_{f_i^{vec}, f_i^c, P_i} \sum_{i=1}^{n} U_i$$
$$s.t. \quad (C1),(C2),(C3),(C4)$$

Although α_i and β_i are decoupled from variables $f_i^c, f_i^{vec}, and P_i$, the problem remains non-smooth due to the non-differentiability of T_i^{total} with respect to f_i^{vec}, f_i^c, and P_i. Therefore, we approximate T_i^{total} as follows:

$$T_i^{total} = \omega_1 D_i^v + \omega_2 (D_i^{vec} + T_i^{vec}) + \omega_3 (T_i^{vec} + T_i^c + D_i^c) \tag{18}$$

Substituting Eqs. (4), (5), (8), (9), and (10) into Eq. (18):

$$\widehat{T_i^{total}} = \frac{(1-\alpha_i-\beta_i)\omega_1 Q_i}{f^v} + \omega_2 \left(\frac{\alpha_i Q_i}{f_i^{vec}} + \frac{(\alpha_i+\beta_i)S_i}{V_i^{vec}}\right) + \omega_3 \left(\frac{(\alpha_i+\beta_i)S_i}{V_i^{vec}} + \frac{\beta_i S_i}{V^c} + \frac{\beta_i Q_i}{f_i^c}\right) \tag{19}$$

where ω_1, ω_2, and ω_3 denote weight factors satisfying $\omega_1 + \omega_2 + \omega_3 = 1$. In contrast to that in [31], the approximate value of T_i^{total} is not simply the sum of all latencies, which would overly emphasize the weight of system latency in this tradeoff. Instead, we use normalized weight factors to provide a more accurate representation of total latency. Substituting Eq. (19) into $P1.1$:

$$P1.2: \max_{f_i^{vec}, f_i^c, P_i} \sum_{i=1}^{n} U_i$$
$$s.t. \quad (C1),(C2),(C4)$$
$$\widehat{T_i^{total}} \leq T_i^{max} \tag{C3a}$$

Lemma 1. *$P1.2$ represents a convex optimization problem.*

The detailed proof of Lemma 1 can be found in the Appendix A. Based on Lemma 1, $P1.2$ can be solved using the convex optimization techniques. In this study, we employ the Lagrangian multiplier method, and the gradient method to find the optimal solution, which then informs the development of an optimal computation resource allocation strategy.

The Lagrange function for $P1.2$ is expressed as :

$$L_i = \widehat{U}_i + \varphi\left(\sum_{i=1}^{n} f_i^{vec} - F^{vec}\right) + \kappa\left(\sum_{i=1}^{n} f_i^c - F^c\right) + \gamma_i(\widehat{T_i^{total}} - T^{max}) + \lambda_i(E_i^{total} - E^{max}) \tag{20}$$

where $\varphi \geq 0$, $\kappa \geq 0$, $\gamma_i \geq 0$, and $\lambda_i \geq 0$ denote the Lagrange multipliers associated with constraints (C1), (C2), (C3a), and (C4), respectively, and $\{f_i^{vec*}, f_i^{c*}, P_i^*\}$ represent the optimal solution of $P1.2$. Taking the partial derivative of L_i with respect to $f_i^{vec*}, f_i^{c*}, P_i^*$:

$$\frac{\partial L_i}{\partial f_i^{vec*}} = \frac{(\theta-\gamma_i)\omega_2\alpha_i Q_i}{(f_i^{vec*})^2} + (\theta+\lambda_i-1)\alpha_i\zeta_i^{vec}Q^2 + \varphi \tag{21}$$

$$\frac{\partial L_i}{\partial f_i^{c*}} = \frac{(\theta-\gamma)\omega_3\beta_i Q_i}{(f_i^{c*})^2} + (\theta+\lambda_i-1)\beta_i\zeta_i^c Q^2 + \kappa \tag{22}$$

$$\frac{\partial L_i}{\partial P_i^*} = -\frac{(\omega_2+\omega_3)(\alpha_i+\beta_i)\gamma_i S_i C_i}{(\log(1+\frac{P_i^* C_i}{N}))^2(1+\frac{P_i^* C_i}{N})BN} + \lambda_i\xi^v \tag{23}$$

Let Eqs. (21), (22), and (23) be equal to 0, the optimal solution can be calculated as:

$$f_i^{vec*} = \sqrt{\frac{(\gamma_i-\theta)\omega_2\alpha_i Q_i}{(\theta+\lambda_i-1)\alpha_i\zeta_i^{vec}Q^2+\varphi}} \tag{24}$$

$$f_i^{c*} = \sqrt{\frac{(\gamma_i-\theta)\omega_3\beta_i Q_i}{(\theta+\lambda_i-1)\beta_i\zeta_i^c Q^2+\kappa}} \tag{25}$$

$$X^{\sqrt{X}} = exp\left(\sqrt{\frac{(\omega_2+\omega_3)(\alpha_i+\beta_i)\gamma_i S_i C_i}{BN\lambda_i\xi^v}}\right) \tag{26}$$

$$X = 1 + \frac{P_i^* C_i}{N} \tag{27}$$

Eq. (26) is transcendental and cannot be solved directly; however, the Python package *sympy* can be used to solve it. Subsequently, we apply the gradient method to update Lagrange multipliers, achieving the optimal solution upon convergence.

4.2. Computation task offloading problem

The variables in this sub-problem are α_i and β_i, with f_i^{vec}, f_i^c, P_i treated as non-negative constants. This problem can be reformulated through simple mathematical transformations:

$$P2: \max_{\alpha_i,\beta_i} \sum_{i=1}^{n} \Lambda_i\alpha_i + \Pi_i\beta_i$$
$$s.t. \quad (C3a),(C4),(C5)$$

$$\Lambda_i = \left(\frac{\omega_1}{f^v} - \frac{\omega_2}{f_i^{vec}}\right)\theta Q_i + (\zeta^v f^v - \zeta^{vec} f_i^{vec})(1-\theta)Q^2 - \frac{(\omega_2+\omega_3)\theta S_i}{V_i^{vec}} - \tag{28}$$

$$\Pi_i = \left(\frac{\omega_1}{f^v} - \frac{\omega_3}{f_i^c}\right)\theta Q_i + (\zeta^v f^v - \zeta^c f_i^c)(1-\theta)Q^2 - \frac{(\omega_2+\omega_3)\theta S_i}{V_i^{vec}} - \frac{\theta\omega_3 S_i}{V^c} \tag{29}$$

where $P2$ is a linear programming (LP) problem that can be solved using multiple methods. We employ the simplex method [32] to solve $P2$; due to space constraints, its details are omitted in this paper. Additionally, we analyze the scenario where the entire task is offloaded solely to the VEC or cloud, as discussed in Lemma 2.

Lemma 2. *The entire task is offloaded to the cloud if $\Pi_i \geq \Lambda_i \geq 0$, and $1 < \min(\frac{\hat{T}^{max}}{d_i^2}, \frac{\hat{E}^{max}}{e_i^2})$. Conversely, the task is offloaded to the VEC if $\Lambda_i \geq \Pi_i \geq 0$, and $1 < \min(\frac{\hat{T}^{max}}{d_i^1}, \frac{\hat{E}^{max}}{e_i^1})$.*

The complete proof of Lemma 2 is provided in Appendix B. $\Pi_i \geq \Lambda_i \geq 0$ indicates that offloading the task to the cloud is more profitable than to the VEC. $1 < \min(\frac{\hat{T}^{max}}{d_i^2}, \frac{\hat{E}^{max}}{e_i^2})$ signifies that both latency constraint and energy constraints are inoperative, implying low latency and energy consumption requirements, where d_i^1 and d_i^2 indicate the latency of offloading the task of vehicle i to the VEC and cloud, while e_i^1 and e_i^2 denote the corresponding energy consumption.

In summary, we employ the simplex method to address the task splitting problem in end-VEC-cloud orchestrated networks. Additionally, we explore scenarios where the entire task is offloaded to either the VEC or the cloud. In these cases, the latency and energy constraints are non-binding, allowing for improved QoS for latency-insensitive and energy-insensitive applications through full task offloading.

4.3. Collaborative task splitting and resource allocation optimization algorithm

Based on the above analysis, the original problem $P1$ can be decoupled into two convex problems, each of which can be solved to obtain

locally optimal solutions. We propose a collaborative task splitting and resource allocation optimization algorithm, which uses these local solutions to achieve a globally approximate optimal solution. The details of CTSRAO are summarized in Algorithm 1.

Algorithm 1 Collaborative task division and resource allocation optimization (CTSRAO)

1: **Input:** Network and energy parameters $V^c, \zeta^c, \zeta^{vec}$; convergence tolerance ϵ; iteration $\tau = 0$; maximum iterations τ^{max}
2: **Output:** $f_i^{vec,\tau}, f_i^{c,\tau}, P_i^\tau, \alpha_i^\tau, \beta_i^\tau$
3: Initialize staring variable α_i^0, β_i^0
4: **while** $|\sum_i^n U_i^\tau - U_i^{\tau-1}| > \epsilon$ and $\tau < \tau^{max}$ **do**
5: Obtain $f_i^{vec,\tau}, f_i^{c,\tau}, P_i^\tau$ by solving $P1.2$
6: Update $f_i^{vec,\tau}, f_i^{c,\tau}, P_i^\tau$ as follows:
7: $f_i^{vec,\tau} = f_i^{vec,\tau-1} + \mu_1(f_i^{vec,\tau} - f_i^{vec,\tau-1})$
8: $f_i^{c,\tau} = f_i^{c,\tau-1} + \mu_2(f_i^{c,\tau} - f_i^{c,\tau-1})$
9: $P_i^\tau = P_i^{\tau-1} + \mu_3(P_i^\tau - P_i^{\tau-1})$
10: Substituting $f_i^{vec,\tau}, f_i^{c,\tau}, P_i^\tau$ into $P2$
11: Checking Lemma2:
12: If $\Pi_i \geq \Lambda_i \geq 0$, $1 < \min(\frac{\hat{T}^{max}}{d_i^2}, \frac{\hat{E}^{max}}{e_i^2})$
13: Set $\alpha_i^\tau = 0, \beta_i^\tau = 1$
14: Else if $\Lambda_i \geq \Pi_i \geq 0$, $1 < \min(\frac{\hat{T}^{max}}{d_i^1}, \frac{\hat{E}^{max}}{e_i^1})$
15: Set $\alpha_i^\tau = 1, \beta_i^\tau = 0$
16: Else
17: Obtain $\alpha_i^\tau, \beta_i^\tau$ by solving $P2$ with simplex method
18: Update $\alpha_i^\tau, \beta_i^\tau$ as follows:
19: $\alpha_i^\tau = \alpha_i^{\tau-1} + \mu_4(\alpha_i^\tau - \alpha_i^{\tau-1})$
20: $\beta_i^\tau = \beta_i^{\tau-1} + \mu_5(\beta_i^\tau - \beta_i^{\tau-1})$
21: $\tau = \tau + 1$
22: **return** $f_i^{vec,\tau}, f_i^{c,\tau}, P_i^\tau, \alpha_i^\tau, \beta_i^\tau$

Step 3 initializes α_i^0 and β_i^0 in the first iteration. The values $\alpha_i^{\tau-1}, \beta_i^{\tau-1}$ are used as inputs for $P1.2$, and the optimal solution $(f_i^{vec,\tau}, f_i^{c,\tau}, P_i^\tau)$ is obtained by solving $P1.2$ using the Lagrange method in step 5. Steps 7–9 update $f_i^{vec,\tau}$, $f_i^{c,\tau}$, and P_i^τ to prevent these values from becoming excessively large or small, ensuring that the algorithm converges at an appropriate rate and avoids trapping the global solution in a locally optimal solution. Next, $f_i^{vec,\tau}, f_i^{c,\tau}$ are used as constants in $P2$. The optimal partial task splitting ratio is determined by solving $P2$ using the simplex method. Steps 11–16 verify whether the conditions of Lemma 2 are met. If they are, the values of α_i, β_i are provided directly without further calculation. Similar to steps 7–9, $\alpha_i^\tau, \beta_i^\tau$ are updated by steps 18–19, where μ_1, μ_2, μ_3, μ_4, and μ_5 are search parameters controlling the convergence rate. Finally, it is checked if the algorithm has converged. If it has, output the values $f_i^{vec,\tau}, f_i^{c,\tau}, P_i^\tau, \alpha_i^\tau, \beta_i^\tau$. If not, then $\tau = \tau + 1$ and the algorithm returns to step 5.

4.4. Complexity analysis

As previously indicated, we employ Lagrange method and gradient method to solve $P1.2$. There are four Lagrange multipliers that need to be updated using the gradient method. Assume that the number of tasks is n and the number of iterations of gradient method is k. The complexity of gradient method is $\mathcal{O}(4 \cdot n \cdot k) = \mathcal{O}(n \cdot k)$. Then, we use simplex method to solve $P2$, where each task contains two variables. The complexity of simplex method is $\mathcal{O}(2^2 \cdot n) = \mathcal{O}(n)$. These two methods are alternately employed until the algorithm converges. Let the total number of alternating iterations be denoted as m. The overall complexity of CTSRAO is $\mathcal{O}(m \cdot (\mathcal{O}(n \cdot k) + \mathcal{O}(n))) = \mathcal{O}(m \cdot n \cdot k)$.

5. Simulation

In this section, we provide numerical results to assess the performance of the proposed CTSRAO algorithm and validate our theoretical analysis. To ensure the practicality and effectiveness of the experiments, we referred to relevant standards, including those from 3GPP and IEEE.

Table 2
Experimental parameters.

Parameter	Value
VEC maximum rate (Mbps)	12
Vehicle computation resource (GHz)	1.35
Cloud computation resource (GHz)	25
Energy coefficient ρ^{vec}	0.12
Energy coefficient ρ^c	0.10
Energy coefficient ζ^{vec}	0.12
Energy coefficient ζ^c	0.08
Noise power(dbm)	−114
Average vehicular velocity v_i (km/h)	80
VEC coverage radius $\mathbb{L}$ (m)	30
Number of vehicles	4 ∼ 12

5.1. Simulation parameters

We considered a 100-meter stretch of straight road covered by a VEC server. Vehicles traveled at a speed following a Gaussian distribution, each equipped with an OBU and a computation task. The required computation resources, input data size, and maximum energy consumption constraint for each computation task were uniformly distributed within the range of U[0.8, 2.1] GHz, U[3, 7.5] MB, and U[2, 3]$J^{\times 10^4}$, respectively. Other experimental parameters are listed in Table 2.

5.2. Performance

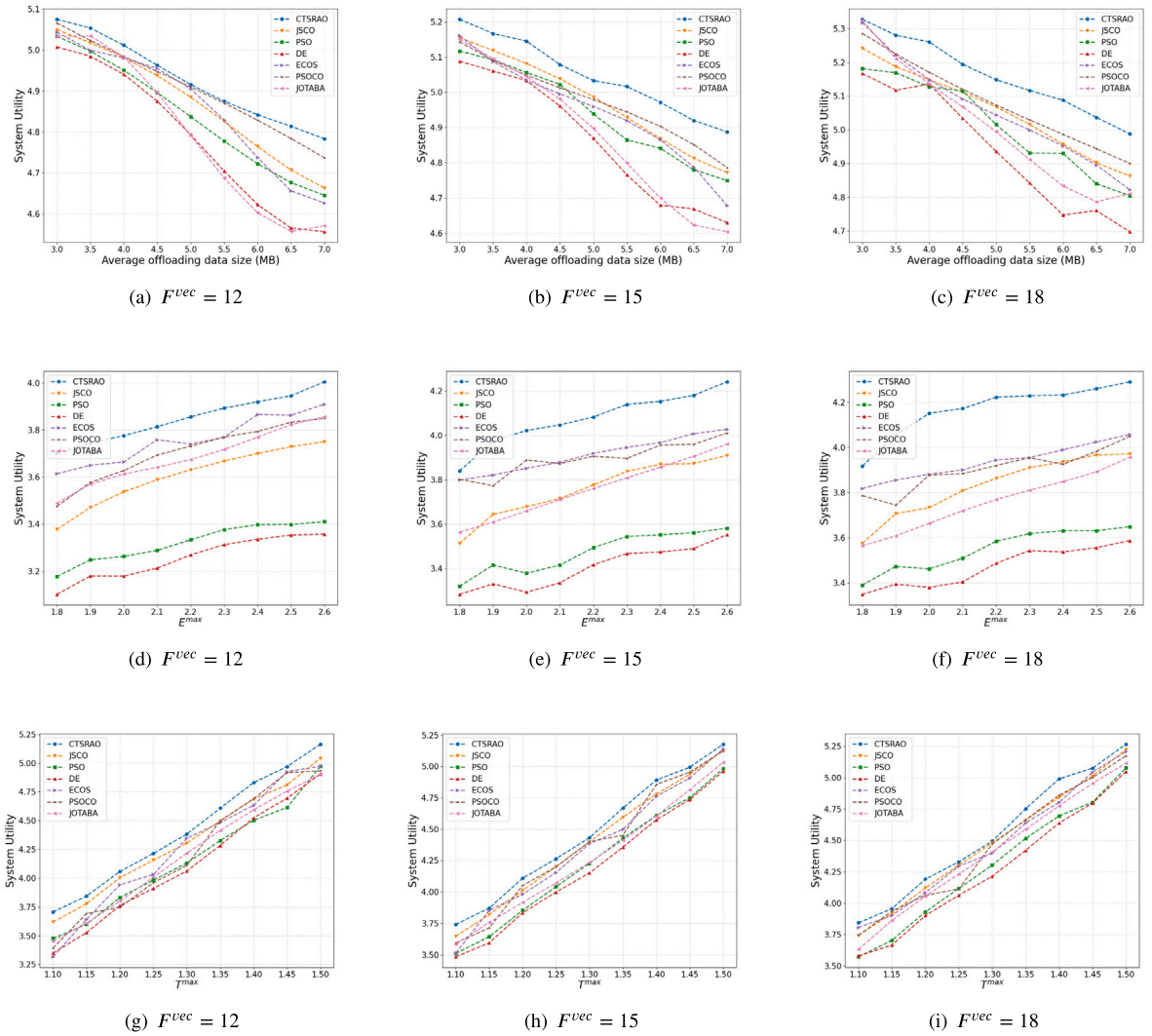
In this section, we evaluate the performance of the proposed CTSRAO algorithm against the state of the art algorithms, including JOTABA [33], JSCO [31], PSOCO [23] and ECOS [21], as well as benchmark algorithms such as particle swarm optimization (PSO) and differential evolution (DE). We also explore the relationship between total latency and total energy consumption across different computation resources and present the convergence results of CTSRAO.

Fig. 3(a)–(c) illustrate the system utility of the compared algorithms across varying offloading data sizes. It is evident that system utility declines as the data size increases. Among other optimization algorithms, CTSRAO demonstrates superior performance and stability in system utility across different data sizes.

Fig. 3(d)–(f) illustrate the effect of the maximum tolerance energy consumption on system utility, with a maximum tolerance latency of 1.1s. System utility increases proportionally with E^{max}, although the growth rate of system utility diminishes as E^{max} grows. The CTSRAO algorithm consistently yields higher system utility than PSOCO and JOTABA within the range of E^{max} from 1.8 to 2.6. Moreover, CTSRAO exhibits the least sensitivity to changes in E^{max}, indicating its effectiveness in enhancing when energy is sufficient. In contrast, compared algorithms show minimal improvement when E^{max} is higher than 2.4 when $F^{vec} = 15$. Furthermore, the system utility of CTSRAO consistently outperforms PSOCO across different E^{max} values.

Fig. 3(g)–(i) depict the effect of maximum tolerance latency on system utility with maximum tolerance energy consumption fixed at 1.8. CTSRAO outperforms other algorithms, with system utility increasing with T^{max}. The performance of PSOCO is unstable especially when $F^{vec} = 18$. In Fig. 3(h), PSO and DE show similar system utility, particularly when T^{max} is between 1.45 to 1.5, due to their convergence on a local optimal solution that limits further improvement through iteration.

Fig. 4(a) illustrates the relationship between the total latency T^{tol} and total energy consumption E^{tol}. The near overlap of the three curves indicates that the total computation resources of the VEC have minimal impact on the relationship between T^{tol} and E^{tol}. Moreover, T^{tol} is inversely proportional to E^{tol}, and the slope of the curve decreases as T^{tol} increases. A high T^{max} renders the latency constraint ineffective, leading the algorithm to minimize E^{tol}. Conversely, if the energy

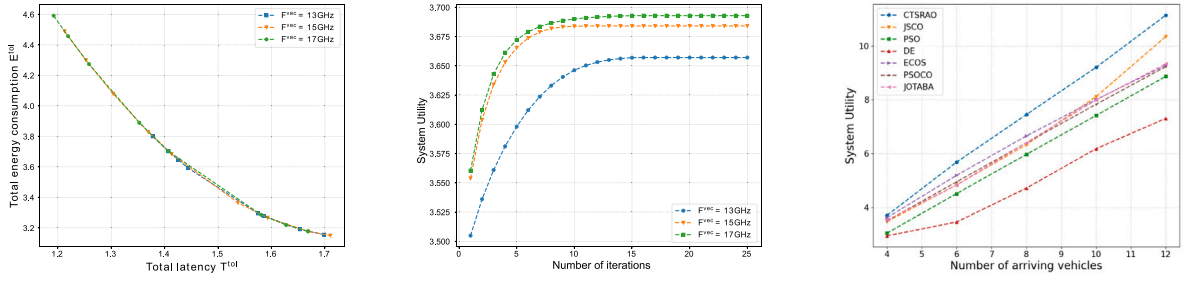


(a) $F^{vec} = 12$ (b) $F^{vec} = 15$ (c) $F^{vec} = 18$

(d) $F^{vec} = 12$ (e) $F^{vec} = 15$ (f) $F^{vec} = 18$

(g) $F^{vec} = 12$ (h) $F^{vec} = 15$ (i) $F^{vec} = 18$

Fig. 3. Comparison of various algorithms.



(a) (b) (c)

Fig. 4. Performance under different configurations.

constraint is ineffective, the algorithm will minimize T^{tol}.

Fig. 4(b) shows that CTSRAO converges after 10 iterations with VEC computation resources of 15 GHz or 17 GHz. However, convergence occurs after 15 iterations with 13 GHz. This delay is due to resource competition among tasks when VEC computation resources are limited, as constrained by (C1). Insufficient VEC resources result in suboptimal system utility.

Fig. 4(c) depicts the effect of varying vehicle numbers on system utility. The CTSARO algorithm consistently outperforms other algorithms across different computation tasks. The system utility gap between CTSARO and these three optimization algorithms widens as the number of vehicles increases; primarily because a larger solution space with more input variables makes it more difficult for evolutionary algorithms to identify the optimal solution. In contrast, the system utility of CTSARO steadily increases with the number of vehicles, demonstrating that CTSARO is more stable and effective than PSOCO, ECOS, JOTABA and JSCO.

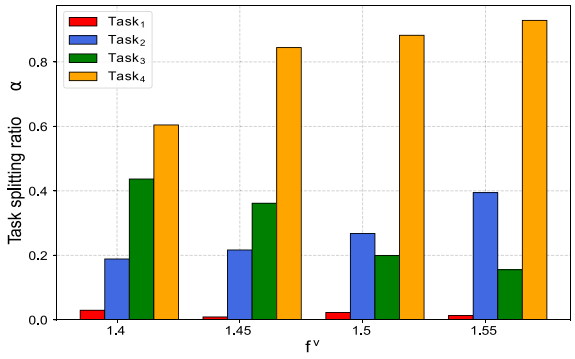


Fig. 5. Effect of local computation resources on task splitting ratio α.

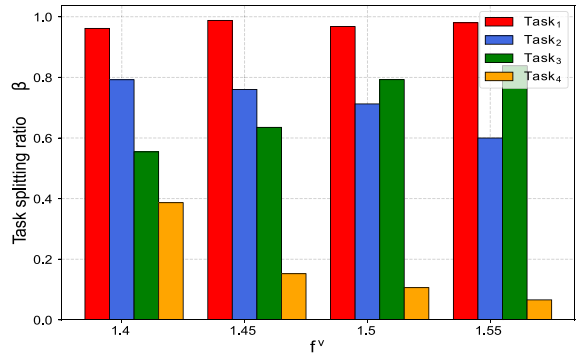


Fig. 6. Effect of local computation resources on task splitting ratio β.

Table 3
Details on various types of tasks.

	D_n	Q_n	T^{max}	E^{max}
Task 1	3000	1.03	0.84	2.12
Task 2	7800	1.84	1.39	2.95
Task 3	9270	2.07	1.58	3.31
Task 4	4000	1.13	0.85	2.31

5.3. Analysis of task splitting ratio

This subsection examines factors affecting the task splitting ratio, including local and VEC computation resources, and the data transfer rate V_c. It also explores system utility under two extreme scheduling scenarios (VEC-only and Cloud-only)

To thoroughly analyze the impact of various factors on the task-splitting ratio, we have defined four test tasks. The details are shown in the table below:

Figs. 5 and 6 illustrate the effect of local computing resources f^v on task splitting ratios α, β for four different tasks. Task 1, being computation-intensive, is fully offloaded to the cloud where α_1, β_1 remain largely unchanged by variations in f^v, as latency and energy constraints of Task 1 are not stringent. Tasks 2 and 4 are latency-sensitive tasks where α_2, α_4 increase with the f^v and β_2, β_4 decrease with f^v. This happens because of diminishing returns of offloading tasks to the cloud as f^v increases, resulting in a higher revenue ratio $\frac{\Lambda_i}{\Pi_i}$. Task 3, being energy-sensitive and computation-intensive, has stringent energy constraints demands greater computation resources. As f^v increases, α_3 decreases, and β_3 increases. Differing from Tasks 2 and 4, the revenue ratio $\frac{\Lambda_i}{\Pi_i}$ for Task 3 decreases with increasing f^v.

To visually assess fluctuations in task splitting ratios across varying conditions, we employ line charts as a means to present our experimental results. Figs. 7(a) and 7(b) reveal that as computation resources increase, Tasks 2 and 4 shift more workload to the VEC. Especially, with 16 computation units, Task 4 nearly offloads all tasks to the VEC, and the task splitting ratio α for Task 2 stabilizes when computation resources exceed 16. However, Task 1 continues to offload all workloads to the cloud despite increases in VEC computation resources, due to its computation-intensive nature and relatively lenient latency requirements. In contrast, Task 3 is latency-sensitive and requires processing on the VEC to meet its stringent latency constraints.

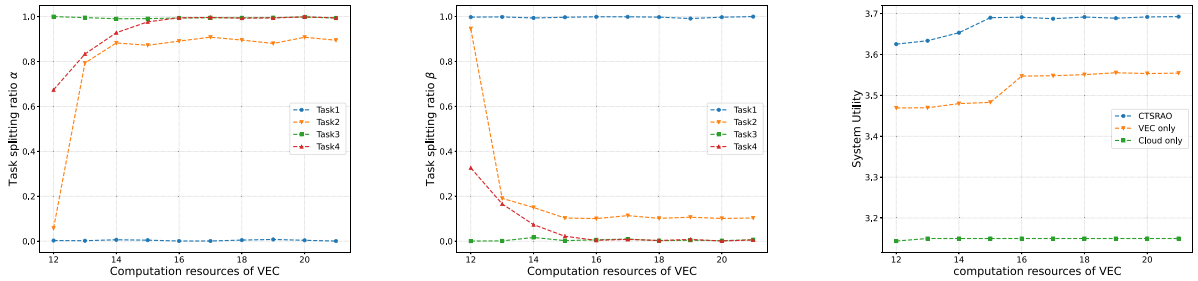
We explore the effect of V_c on task-splitting ratios. Figs. 8(a) and 8(b) reveal that when V_c exceeds 1.3 MB/s, Tasks 2 and 4 progressively shift more workload to the cloud, underscoring the significant impact of V_c on workload distribution. A distinctive observation emerges as shown in Fig. 7(a), where Tasks 2 and 4 initially favor allocating workload to the VEC. However, Fig. 8(a) reveals a shift toward cloud computing as V_c increases, reflecting dynamic workload adjustments based on data transfer speed. Latency-sensitive Task 3 consistently allocates all workload to the VEC regardless of V_c, reflecting a strong preference for VEC allocation due to its stringent latency requirements. In summary, the analysis of V_c unveils a dynamic relationship between data transfer speed and workload distribution, emphasizing the need for nuanced decision-making and resource allocation strategies in end-VEC-cloud networks.

To highlight the importance of task splitting, we compare our method with two scheduling strategies: exclusive VEC scheduling and exclusive scheduling to the cloud central. We also examine the system utility of these scheduling strategies under different F^{vec} and V_c. As shown in Fig. 7(c), the system utility of the VEC-only scheduling improves gradually with increasing VEC computational resources, experiencing a notable rise when resources reach 16. However, system utility remains largely unchanged when F^{vec} exceeds 16, as the task's computational needs are met and additional resources only increase energy consumption without improving system utility. Since cloud-only scheduling is unaffected by changes in VEC resources, its system utility remains the lowest. Overall, the CTSRAO approach, which optimizes task splitting ratio, significantly outperforms the other two scheduling schemes.

Fig. 8(c) presents system efficiency variations across the three scheduling schemes with respect to the data transfer rate V_c. Notably, the system utility for the cloud-only scheduling improves with increasing V_c. The system utility of Cloud-only scheduling is primarily influenced by V_c with a faster data transfer speed enhancing the system utility. Similarly, V_c has no impact on the system utility of VEC-only scheduling. Notably, CTSRAO experiences slightly reduced utility when the data transfer rate A exceeds 1.35 MB/s. This phenomenon occurs because CTSRAO attempts to shift a greater proportion of Tasks 2 and 4 to the cloud, aiming to meet latency requirements while achieving cost savings, leading to a slight decline in system utility. Due to the large tradeoff coefficient θ in this experiment, the effect of energy conservation on system utility is minimal. Nonetheless, our algorithm consistently outperforms the two extreme scheduling approaches, regardless of changes in the data transfer rate V_c.

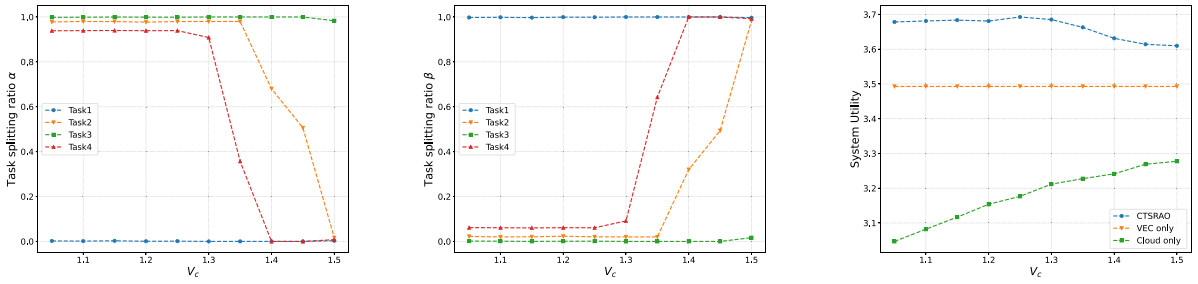
5.4. Ablation study

To comprehensively assess the performance of our algorithm, we conducted a thorough ablation study, comparing CTSRAO with four ablation algorithms. Specifically, CTSRAO-A evenly allocates VEC resources to different tasks, while CTSRAO-R randomly partitions tasks. The VEC-only approach offloads the entire task to VEC, and the Cloud-only approach offloads the entire task to the cloud. As illustrated in Fig. 9, our algorithm demonstrates the best performance among all algorithms, significantly surpassing the other ablation algorithms when $F^{vec} = 15$ and $F^{vec} = 18$. Box plot analysis can reveal the degree of data dispersion. The values of CTSRAO exhibit greater concentration in Fig. 9, indicating a higher level of performance stability. CTSRAO-R exhibit considerable fluctuation, which may contribute to unstable outcomes. The performance of CTSRAO-A in the ablation study is



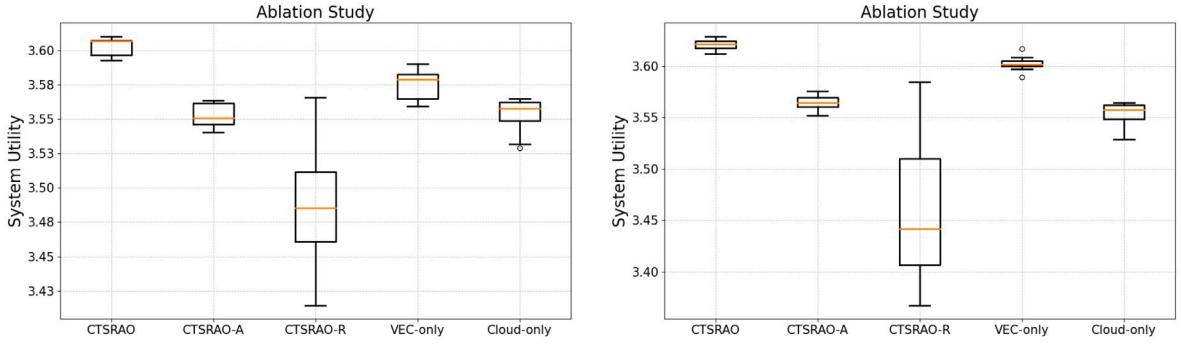
(a) (b) (c)

Fig. 7. Splitting ratio and performance of tasks with different VEC computation resources.



(a) (b) (c)

Fig. 8. Task performance and splitting ratio for varying V_c.



(a) $F^{vec} = 15$ (b) $F^{vec} = 18$

Fig. 9. Ablation study.

inferior to that of VEC-only, which demonstrates the importance of the resource allocation optimization module within the CTSRAO. Based on the above analysis, our algorithm outperforms the ablation algorithms in both performance and stability.

6. Conclusion

This paper introduced a novel optimization approach for end-VEC-cloud orchestrated networks, addressing the joint computation resource allocation and computing offloading to enhance system utility. Our approach systematically addressed the optimization goal by minimizing total energy consumption and latency, which are critical to network performance. We decoupled this complex challenge into two convex sub-problems, facilitating a more streamlined solution. Subsequently, we introduced the CTSRAO algorithm, an innovative approach for optimizing resource allocation and task-splitting ratios. This algorithm identified the optimal resource allocation strategy and task-splitting ratios. Extensive simulations highlighted the remarkable performance of CTSRAO, consistently surpassing existing optimization algorithms in reducing total latency and energy consumption. This novel technique also showed superior stability and efficiency across varying numbers of computation tasks, underscoring its robustness and adaptability in diverse network scenarios. In a nutshell, the CTSRAO algorithm has emerged as an effective tool for optimizing computation resource allocation and computing offloading in end-VEC-cloud orchestrated networks. Its superior performance enabled significant advancement in network optimization, offering enhanced efficiency and stability across various computational scenarios.

CRediT authorship contribution statement

Yuxuan Long: Writing – review & editing, Writing – original draft, Resources, Methodology, Formal analysis, Conceptualization. **Zhenyu Wang:** Funding acquisition. **Shizhan Lan:** Visualization. **Rui Zhang:** Supervision. **Kai Xu:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Proof of Lemma 1

Proof. According to duality theory, the max-concave optimization problem is convex. To prove that $P1.2$ is convex, we must show that the objective function U_i is concave. By substituting Eqs. (17) and (19) into Eq. (15), U_i is expressed as:

$$\begin{aligned}\widehat{U}_i = \Gamma &- \frac{\theta\omega_2\alpha_i Q_i}{f_i^{vec}} - \frac{\theta\omega_3\beta_i Q_i}{f_i^c} - (1-\theta)\alpha_i\zeta^{vec} f_i^{vec} Q^2 \\ &- (1-\theta)\beta_i\zeta^c f_i^c Q^2 - \frac{(\omega_2+\omega_3)(\alpha_i+\beta_i)\theta S_i}{V_i^{vec}} \\ &- (1-\theta)\xi^v P_i \end{aligned} \tag{30}$$

$$\begin{aligned}\Gamma = \theta T_i^{max} &+ (1-\theta)E_i^{max} - \frac{\theta\omega_3\beta_i S_i}{V_c} - (1-\theta)(\xi^c P^c) \\ &- (1-\alpha_i-\beta_i)\left(\frac{\theta\omega_1 Q_i}{f^v} + (1-\theta)\zeta^v f^v Q_i^2\right)\end{aligned} \tag{31}$$

Next, we examine the Hessian of $\widehat{U}_i$:

$$H = \begin{bmatrix} \frac{\partial^2\widehat{U}_i}{\partial(f_i^{vec})^2} & \frac{\partial^2\widehat{U}_i}{\partial f_i^{vec}\partial f_i^c} & \frac{\partial^2\widehat{U}_i}{\partial f_i^{vec}\partial P_i} \\ \frac{\partial^2\widehat{U}_i}{\partial f_i^c\partial f_i^{vec}} & \frac{\partial^2\widehat{U}_i}{\partial(f_i^c)^2} & \frac{\partial^2\widehat{U}_i}{\partial f_i^c\partial P_i} \\ \frac{\partial^2\widehat{U}_i}{\partial P_i\partial f_i^{vec}} & \frac{\partial^2\widehat{U}_i}{\partial P_i\partial f_i^c} & \frac{\partial^2\widehat{U}_i}{\partial(P_i)^2} \end{bmatrix} \tag{32}$$

All leading principal minors can be determined through a mathematical calculation, as shown below:

$$\Delta_1 = -\frac{\theta\omega_2\alpha_i Q_i}{(f_i^{vec})^3} < 0 \tag{33}$$

$$\Delta_2 = \frac{\omega_2\omega_3\alpha_i\beta_i\theta^2 Q_i^2}{(f_i^{vec} f_i^c)^3} > 0 \tag{34}$$

$$\Delta_3 = -\frac{(\omega_2+\omega_3)(\alpha_i+\beta_i)(2+\log(1+\frac{P_i C_i}{N}))C_i^2\theta S_i\Delta_2}{B(\log(1+\frac{P_i C_i}{N}))^3(N+P_i C_i)^2} \tag{35}$$

$$\Delta_3 < 0$$

Since the first-order leading principal minor H_{11} is negative, the second-order minor Δ_2 is positive, and the third-order minor Δ_3 is negative. Therefore, the Hessian matrix H is negative definite. Consequently, U_i is concave on both f_i^c, f_i^{vec}, P_i, making $P1.2$ a max-concave and therefore convex problem.

Appendix B. Proof of Lemma 2

Proof. This proof utilizes the simplex method, which is not detailed in this paper. Constraint (C3a) can be reformulated through simple mathematical transformations:

$$d_i^1\alpha_i + d_i^2\beta_i \le T^{max} - \frac{\omega_1 Q_i}{f^v} \tag{C3b}$$

$$d_i^1 = \frac{\omega_2 Q_i}{f_i^{vec}} - \frac{\omega_1 Q_i}{f^v} + \frac{(\omega_2+\omega_3)S_i}{V_i^{vec}} \tag{36}$$

$$d_i^2 = \left(\frac{\omega_2 S_i}{V_i^{vec}} - \frac{\omega_1 Q_i}{f^v} + \omega_3\left(\frac{S_i}{V_c} + \frac{Q_i}{f_i^c} + \frac{S_i}{V_i^{vec}}\right)\right) \tag{37}$$

Similarly, constraint (C4) can be reformulated through simple mathematical transformations:

$$e_i^1\alpha_i + e_i^2\beta_i \le E^{max} - \zeta^v f^v Q_i^2 \tag{C4a}$$

$$e_i^1 = (\zeta^{vec} f_i^{vec} - \zeta^v f^v)Q_i^2 \tag{38}$$

$$e_i^2 = (\zeta^c f_i^c - \zeta^v f^v)Q_i^2 \tag{39}$$

Since the tasks are linearly independent, Lemma 2 can be demonstrated for a single-task scenario. To apply the simplex method to solve $P2$, slack variables $x_3, x_4, x_5 \ge 0$ must be introduced to convert $P2$ into a standard form:

$$P2.1: \min_{\alpha_i,\beta_i} -\Lambda_i\alpha_i - \Pi_i\beta_i$$

$$s.t. \quad d_i^1\alpha_i + d_i^2\beta_i + x_3 = \widehat{T}^{max} \tag{C3c}$$

$$e_i^1\alpha_i + e_i^2\beta_i + x_4 = \widehat{E}^{max} \tag{C4b}$$

$$\alpha_i + \beta_i + x_5 = 1 \tag{C5a}$$

where $\widehat{T}^{max} = T^{max} - \frac{\omega_1 Q_i}{f^v}$ and $\widehat{E}^{max} = E^{max} - \zeta^v f^v Q_i^2$. According to [32], the constraints can be expressed as $Ax = b$, where A denotes the coefficient matrix, $x = [\alpha_i, \beta_i, x_3, x_4, x_5]^T$, and $b = [\widehat{T}^{max}, \widehat{E}^{max}, 1]^T$.

$$A = (p_1, p_2, p_3, p_4, p_5) = \begin{bmatrix} d_i^1 & d_i^2 & 1 & 0 & 0 \\ e_i^1 & e_i^2 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$$Let \quad B = (p_3, p_4, p_5) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Next, we initialize a feasible solution:
$x = [0, 0, \widehat{T}^{max}, \widehat{E}^{max}, 1]^T$.

$$Let \quad x = \begin{bmatrix} x_B \\ x_N \end{bmatrix}, x_B = \begin{bmatrix} x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} \widehat{T}^{max} \\ \widehat{E}^{max} \\ 1 \end{bmatrix}, x_N = \begin{bmatrix} \alpha_i \\ \beta_i \end{bmatrix}$$

The objective optimization function $f = -\Lambda_i\alpha_i - \Pi_i\beta_i$ can be expressed as:

$$f = cx = (c_B, c_N)\begin{bmatrix} x_B \\ x_N \end{bmatrix} \tag{40}$$

In the first iteration, substituting x_N into Eq. (40) yields the objective function value $f_1 = 0$. The value of c_B can be determined using Eq. (41):

$$f_1 = c_B x_B = 0 \tag{41}$$

$$c_B = (0, 0, 0)$$

The simplex multiplier w is computed using the following formulation:

$$w = c_B B^{-1} = (0,0,0)\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = (0,0,0) \tag{42}$$

The convergence conditions are determined as follows:

$$z_1 - c_1 = wp_1 - c_1 = (0,0,0)(d_i^1, e_i^1, 1)^T + \Lambda_i = \Lambda_i \tag{43}$$

$$z_2 - c_2 = wp_2 - c_2 = (0,0,0)(d_i^2, e_i^2, 1)^T + \Pi_i = \Pi_i \tag{44}$$

where $z_k - c_k$ denotes the reduction in the objective function, and the basic solution x_N and c_k signify the coefficients of x in the objective function f. Given that offloading tasks to the cloud is generally more beneficial than to the VEC, we assume that $\Pi_i \ge \Lambda_i \ge 0$. Therefore, we can identify the number of subscript k and compute y_k:

$$z_k - c_k = \max\{z_j - c_j\} = z_2 - c_2 = \Pi_i, k = 2 \tag{45}$$

$$y_k = y_2 = B^{-1}p_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} d_i^2 \\ e_i^2 \\ 1 \end{bmatrix} = \begin{bmatrix} d_i^2 \\ e_i^2 \\ 1 \end{bmatrix} \tag{46}$$

$$\overline{b} = x_B = (\widehat{T}^{max}, \widehat{E}^{max}, 1)^T \tag{47}$$

$$\frac{\overline{b_r}}{y_{r2}} = min(\frac{\overline{b_1}}{y_{12}}, \frac{\overline{b_2}}{y_{22}}, \frac{\overline{b_3}}{y_{32}}) = min(\frac{\widehat{T}^{max}}{d_i^2}, \frac{\widehat{E}^{max}}{e_i^2}, 1) \tag{48}$$

where y_{ik} must be less than or equal to 0; otherwise, x_k can assume any positive value, indicating a non-unique optimal solution. If $\widehat{T^{max}}, \widehat{E^{max}}$ are indeterminate in sign, set $\frac{\overline{b_r}}{y_{r2}} = \frac{\overline{b_3}}{y_{32}} = 1$, where $1 < \min(\frac{\widehat{T}^{max}}{d_i^2}, \frac{\widehat{E}^{max}}{e_i^2})$, assuming $\frac{\widehat{T}^{max}}{d_i^2}, \frac{\widehat{E}^{max}}{e_i^2} > 0$. With $k = 2$, a new B matrix is generated replacing p_5 with p_2:

$$B = (p_3, p_4, p_2) = \begin{bmatrix} 1 & 0 & d_i^2 \\ 0 & 1 & e_i^2 \\ 0 & 0 & 1 \end{bmatrix},$$

$$B^{-1} = \begin{bmatrix} 1 + \frac{d_i^2}{e_i^2} & 0 & -d_i^2 \\ 0 & 1 & e_i^2 \\ 0 & 0 & 1 \end{bmatrix} \tag{49}$$

We proceed to the second iteration and follow the previously outlined steps:

$$x_B = \begin{bmatrix} x_3 \\ x_4 \\ \beta_i \end{bmatrix} = \begin{bmatrix} \hat{T}^{max} - d_i^2 \\ \hat{E}^{max} - e_i^2 \\ 1 \end{bmatrix}, x_N = \begin{bmatrix} \alpha_i \\ x_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$f_2 = c_B x_B = -\Pi_i, c_B = (0, 0, -\Pi_i)$$
$$w = c_B B^{-1} = (0, 0, -\Pi_i)$$

The convergence condition is validated as:

$$z_1 - c_1 = w p_1 - c_1 = \Lambda_i - \Pi_i < 0 \tag{50}$$

$$z_5 - c_5 = w p_5 - c_5 = -\Pi_i < 0 \tag{51}$$

Since all $\{z_j - c_j\} \leq 0$, the algorithm has converged, and the basic solutions $\alpha_i = 0, \beta_i = 1$ is optimal. Similarly, if $\Lambda_i \geq \Pi_i \geq 0$ and $1 < \min(\frac{\hat{T}^{max}}{d_i^1}, \frac{\hat{E}^{max}}{e_i^1})$, then $\alpha_i = 1, \beta_i = 0$ is the optimal solution.

Data availability

The authors do not have permission to share data.

References

[1] Y. Yang, K. Hua, Emerging technologies for 5G-enabled vehicular networks, IEEE Access 7 (2019) 181117–181141.

[2] X. Xu, Y. Xue, X. Li, L. Qi, S. Wan, A computation offloading method for edge computing with vehicle-to-everything, IEEE Access 7 (2019) 131068–131077.

[3] S. Li, N. Zhang, H. Chen, S. Lin, O.A. Dobre, H. Wang, Joint road side units selection and resource allocation in vehicular edge computing, IEEE Trans. Veh. Technol. (2021).

[4] S. Wang, G. Chen, Y. Jiang, X. You, A cluster-based V2V approach for mixed data dissemination in urban scenario of IoVs, IEEE Trans. Veh. Technol. 72 (3) (2022) 2907–2920.

[5] W. Feng, N. Zhang, S. Li, S. Lin, R. Ning, S. Yang, Y. Gao, Latency minimization of reverse offloading in vehicular edge computing, IEEE Trans. Veh. Technol. 71 (5) (2022) 5343–5357.

[6] J. Ren, G. Yu, Y. He, G.Y. Li, Collaborative cloud and edge computing for latency minimization, IEEE Trans. Veh. Technol. 68 (5) (2019) 5031–5044.

[7] X. Huang, K. Xu, C. Lai, Q. Chen, J. Zhang, Energy-efficient offloading decision-making for mobile edge computing in vehicular networks, EURASIP J. Wireless Commun. Networking 2020 (1) (2020) 1–16.

[8] Z. Zhou, J. Feng, Z. Chang, X. Shen, Energy-efficient edge computing service provisioning for vehicular networks: A consensus ADMM approach, IEEE Trans. Veh. Technol. 68 (5) (2019) 5087–5099.

[9] K. Behravan, N. Farzaneh, M. Jahanshahi, S.A.H. Seno, A comprehensive survey on using fog computing in vehicular networks, Veh. Commun. (2023) 100604.

[10] J. Huang, W. Wu, W. Huang, Y. Xiao, L. Lisi, J. Sun, A survey on task partitioning and scheduling for vehicular edge computing, in: 2023 IEEE 10th International Conference on Cyber Security and Cloud Computing (CSCloud)/2023 IEEE 9th International Conference on Edge Computing and Scalable Cloud, EdgeCom, IEEE, 2023, pp. 336–342.

[11] S. Raza, S. Wang, M. Ahmed, M.R. Anwar, A survey on vehicular edge computing: architecture, applications, technical issues, and future directions, Wirel. Commun. Mob. Comput. 2019 (2019).

[12] L. Liu, C. Chen, Q. Pei, S. Maharjan, Y. Zhang, Vehicular edge computing and networking: A survey, Mob. Netw. Appl. 26 (3) (2021) 1145–1168.

[13] J. Zhao, Q. Li, Y. Gong, K. Zhang, Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks, IEEE Trans. Veh. Technol. 68 (8) (2019) 7944–7956.

[14] F. Sun, F. Hou, N. Cheng, M. Wang, H. Zhou, L. Gui, X. Shen, Cooperative task scheduling for computation offloading in vehicular cloud, IEEE Trans. Veh. Technol. 67 (11) (2018) 11049–11061.

[15] M. Li, P. Si, Y. Zhang, Delay-tolerant data traffic to software-defined vehicular networks with mobile edge computing in smart city, IEEE Trans. Veh. Technol. 67 (10) (2018) 9073–9086.

[16] M. Kanwal, A.W. Malik, A.U. Rahman, I. Mahmood, M. Shahzad, Sustainable vehicle-assisted edge computing for big data migration in smart cities, IEEE Internet Things J. 7 (3) (2019) 1857–1871.

[17] J. Tang, S. Liu, L. Liu, B. Yu, W. Shi, Lopecs: A low-power edge computing system for real-time autonomous driving services, IEEE Access 8 (2020) 30467–30479.

[18] T. Bahreini, M. Brocanelli, D. Grosu, Energy-aware resource management in vehicular edge computing systems, in: 2020 IEEE International Conference on Cloud Engineering, IC2E, IEEE, 2020, pp. 49–58.

[19] X. Peng, Z. Han, W. Xie, C. Yu, P. Zhu, J. Xiao, J. Yang, Deep reinforcement learning for shared offloading strategy in vehicle edge computing, IEEE Syst. J. 17 (2) (2022) 2089–2100.

[20] J. Zhang, X. Hu, Z. Ning, E.C.-H. Ngai, L. Zhou, J. Wei, J. Cheng, B. Hu, Energy-latency tradeoff for energy-aware offloading in mobile edge computing networks, IEEE Internet Things J. 5 (4) (2017) 2633–2645.

[21] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, S. Yu, Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing, IEEE Trans. Veh. Technol. 69 (12) (2020) 14198–14211.

[22] P. Cai, F. Yang, J. Wang, X. Wu, Y. Yang, X. Luo, Jote: Joint offloading of tasks and energy in fog-enabled iot networks, IEEE Internet Things J. 7 (4) (2020) 3067–3082.

[23] Q. Luo, C. Li, T.H. Luan, W. Shi, Minimizing the delay and cost of computation offloading for vehicular edge computing, IEEE Trans. Serv. Comput. 15 (5) (2021) 2897–2909.

[24] M. Qin, N. Cheng, Z. Jing, T. Yang, W. Xu, Q. Yang, R.R. Rao, Service-oriented energy-latency tradeoff for iot task partial offloading in mec-enhanced multi-rat networks, IEEE Internet Things J. 8 (3) (2020) 1896–1907.

[25] L. Bréhon-Grataloup, R. Kacimi, A.-L. Beylot, Mobile edge computing for V2X architectures and applications: A survey, Comput. Netw. 206 (2022) 108797.

[26] M.N. Ahangar, Q.Z. Ahmed, F.A. Khan, M. Hafeez, A survey of autonomous vehicles: Enabling communication technologies and challenges, Sensors 21 (3) (2021) 706.

[27] S. Yousefi, E. Altman, R. El-Azouzi, M. Fathy, Analytical model for connectivity in vehicular ad hoc networks, IEEE Trans. Veh. Technol. 57 (6) (2008) 3341–3356.

[28] Y. AlNagar, S. Hosny, A.A. El-Sherif, Towards mobility-aware proactive caching for vehicular ad hoc networks, in: 2019 IEEE Wireless Communications and Networking Conference Workshop, WCNCW, IEEE, 2019, pp. 1–6.

[29] Z. Yu, J. Hu, G. Min, Z. Zhao, W. Miao, M.S. Hossain, Mobility-aware proactive edge caching for connected vehicles using federated learning, IEEE Trans. Intell. Transp. Syst. 22 (8) (2020) 5341–5351.

[30] C. Xian, Y.-H. Lu, Z. Li, Dynamic voltage scaling for multitasking real-time systems with uncertain execution time, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 27 (8) (2008) 1467–1478.

[31] Y. Dai, D. Xu, S. Maharjan, Y. Zhang, Joint load balancing and offloading in vehicular edge computing and networks, IEEE Internet Things J. 6 (3) (2018) 4377–4387.

[32] G. Dantzig, Linear Programming and Extensions, Princeton University Press, 2016.

[33] N. Zhang, S. Liang, K. Wang, Q. Wu, A. Nallanathan, Computation efficient task offloading and bandwidth allocation in VEC networks, IEEE Trans. Veh. Technol. (2024).

Yuxuan Long received the B.E. degree in computer science and technology from the Zhongnan University of Economics and Law, in 2017, the M.S. degree from Guangxi University, in 2019. He is currently pursuing the Ph.D. degree with the School of Software Engineering, South China University of Technology, China. His current research interests mainly include mobile edge computing and machine learning.

Zhenyu Wang received the Ph.D. degree from the Department of Computer Science, Harbin Institute of Technology, in 1993. He has been a Professor with the South China University of Technology, since 2007. He is currently the Director of the Chinese Information Community of China, and also the Director of the Guangdong Provincial Social Media Processing and Engineering Center. His research interests include natural language processing, text mining, and social network analysis. He has published more than

80 articles. He received the IBM-Ministry of Education University Cooperation Project Excellent Teacher Awards several times, guiding students to win the first and second prizes of the IBM National Competition.

Shizhan Lan received the B.E. degree in communication engineering from the National University of Defense Technology, in 2003, the M.S. degree in Communication and information system from the Communication University of China, in 2008. He is a senior engineer of China Mobile Group Guangxi Co., Ltd, and is currently pursuing the Ph.D. degree with the School of Software Engineering, South China University of Technology, China. His current research interests mainly include Computing Power Network and Mobile Edge Computing.

Rui Zhang received the B.S. and Ph.D. degree from the School of Software Engineering, South China University of Technology, Guangzhou, China, in 2015 and 2021, respectively. His research interests include natural language generation, text mining, and sentiment analysis.

Kai Xu received the M.S. degree in computer technology from Shantou University, China, in 2020. He is currently pursuing the Ph.D. degree with the School of Software Engineering, South China University of Technology, China. His research interests include, dialog systems, convex optimization, cloud computing, and data mining.